P2P 보안 메커니즘

개요

CometBFT P2P 네트워킹에서 보안은 분산 블록체인 시스템의 무결성과 신뢰성을 보장하는 핵심 요소입니다. 악의적인 노드의 공격, 네트워크 도청, 데이터 변조 등의 위협으로부터 시스템을 보호하기 위해 다층적인 보안 메커니즘을 구현합니다.

이 문서에서는 다음 보안 기능들을 다룹니다:

- 암호화된 통신 (Secret Connection)
- 노드 인증 및 신원 확인
- 악의적 피어 탐지 및 차단
- 네트워크 레벨 보안 정책
- DoS/DDoS 방어 메커니즘

암호화된 통신 (Secret Connection)

1. Secret Connection 개요

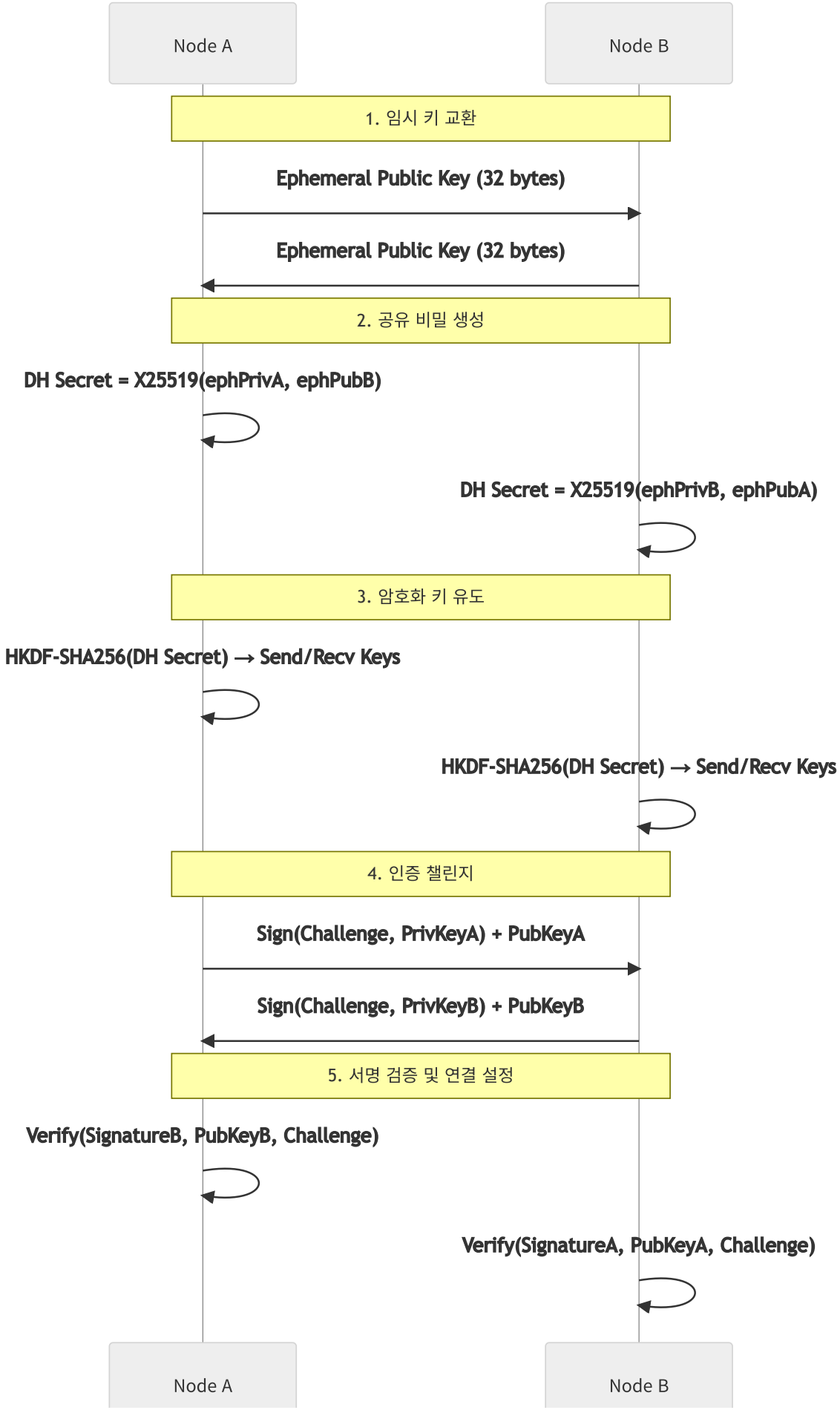
Secret Connection은 CometBFT에서 사용하는 종단간 암호화 프로토콜입니다. STS(Station-to-Station) 프로토콜을 기반으로 구현되어 있습니다.

주요 특징

- **완전 순방향 비밀성 (Perfect Forward Secrecy)**: 세션별 임시 키 사용
- **상호 인증**: 양방향 노드 신원 확인
- **메시지 무결성**: 데이터 변조 방지

- **재생 공격 방지**: Nonce 기반 메시지 순서 보장

2. 암호화 핸드셰이크 과정


Node A
Node B
1. 임시 키 교환
Ephemeral Public Key (32 bytes)
Ephemeral Public Key (32 bytes)
2. 공유 비밀 생성
DH Secret = X25519(ephPrivA, ephPubB)
DH Secret = X25519(ephPrivB, ephPubA)
3. 암호화 키 유도
HKDF-SHA256(DH Secret) → Send/Recv Keys
HKDF-SHA256(DH Secret) → Send/Recv Keys
4. 인증 챌린지
Sign(Challenge, PrivKeyA) + PubKeyA
Sign(Challenge, PrivKeyB) + PubKeyB
5. 서명 검증 및 연결 설정
Verify(SignatureB, PubKeyB, Challenge)
Verify(SignatureA, PubKeyA, Challenge)
Node A
Node B

3. 암호화

알고리즘

키 교환: X25519 (Curve25519)

```
func computeDHSecret(remPubKey, locPrivKey *[32]byte) (*[32]byte, error) {
shrKey, err := curve25519.X25519(locPrivKey[:], remPubKey[:])
if err != nil {
return nil, err
}
var shrKeyArray [32]byte
copy(shrKeyArray[:], shrKey)
return &shrKeyArray, nil
}
```

대칭 암호화: ChaCha20-Poly1305

- **암호화**: ChaCha20 스트림 암호
- **인증**: Poly1305 MAC (Message Authentication Code)
- **키 크기**: 32바이트
- **Nonce 크기**: 12바이트

키 유도: HKDF-SHA256

```go
func deriveSecrets(dhSecret *[32]byte, locIsLeast bool) (recvSecret, sendSecre
hash := sha256.New
hkdf := hkdf.New(hash, dhSecret[:], nil, secretConnKeyAndChallengeGen)
res := new([2*aeadKeySize + 32]byte)
io.ReadFull(hkdf, res[:])
recvSecret = new([aeadKeySize]byte)
sendSecret = new([aeadKeySize]byte)
if locIsLeast {
copy(recvSecret[:], res[0:aeadKeySize])
copy(sendSecret[:], res[aeadKeySize:aeadKeySize*2])
} else {
copy(sendSecret[:], res[0:aeadKeySize])
copy(recvSecret[:], res[aeadKeySize:aeadKeySize*2])
}
return recvSecret, sendSecret
}
```

4. 프레임 암호화

메시지 프레임 구조

```
+----------+------------------+
|  Length |   Encrypted Data |
```

```
| (4 bytes)| (up to 1024 bytes)|
+----------+------------------+
```

암호화 과정

```go
func (sc *SecretConnection) Write(data []byte) (n int, err error) {
for 0 < len(data) {
chunk := data[:min(dataMaxSize, len(data))]
// 프레임 구성: [길이][데이터]
binary.LittleEndian.PutUint32(frame, uint32(len(chunk)))
copy(frame[dataLenSize:], chunk)
// ChaCha20-Poly1305로 암호화
sc.sendAead.Seal(sealedFrame[:0], sc.sendNonce[:], frame, nil)
incrNonce(sc.sendNonce)
sc.connWriter.Write(sealedFrame)
data = data[len(chunk):]
}
return n, nil
}
```

노드 인증 및 신원 확인

1. 노드 신원 체계

Node ID 생성

```go
// Node ID는 공개키의 SHA256 해시의 처음 20바이트
func PubKeyToID(pubKey crypto.PubKey) ID {
return ID(pubKey.Address())
}

// Address는 공개키의 SHA256 해시의 처음 20바이트
func (pubKey Ed25519PubKey) Address() Address {
hasher := sha256.New()
hasher.Write(pubKey[:])
return Address(hasher.Sum(nil)[:AddressSize])
}
```

NodeInfo 구조

```go
type DefaultNodeInfo struct {
ProtocolVersion ProtocolVersion
DefaultNodeID ID
ListenAddr string
Network string
Version string
Channels []byte
Moniker string
Other DefaultNodeInfoOther
```

```
}
```

2. 핸드셰이크 인증 과정

NodeInfo 교환

```
func exchangeNodeInfo(ourNodeInfo ni.NodeInfo, s transport.HandshakeStream, tim
// 동시에 NodeInfo 송수신
go func() {
ourNodeInfoProto := ourNodeInfo.(ni.Default).ToProto()
protoio.NewDelimitedWriter(s).WriteMsg(ourNodeInfoProto)
}()
go func() {
var pbpeerNodeInfo tmp2p.DefaultNodeInfo
protoio.NewDelimitedReader(s, ni.MaxSize()).ReadMsg(&pbpeerNodeInfo)
}()
return peerNodeInfo, nil
}
```

호환성 검증

```
func (info DefaultNodeInfo) CompatibleWith(other NodeInfo) error {
// 네트워크 ID 확인
if info.Network != other.Network {
```

```
return fmt.Errorf("peer is on a different network. Got %v, expected %v",
other.Network, info.Network)
}
// 프로토콜 버전 확인
if !info.ProtocolVersion.IsCompatibleWith(other.ProtocolVersion) {
return fmt.Errorf("peer protocol version mismatch")
}
return nil
}
```

3. 디지털 서명 검증

챌린지 서명

```
func signChallenge(challenge *[32]byte, locPrivKey crypto.PrivKey) ([]byte, err
signature, err := locPrivKey.Sign(challenge[:])
if err != nil {
return nil, err
}
return signature, nil
}
```

서명 검증

```
func verifyAuthSignature(pubKey crypto.PubKey, challenge []byte, signature []by
return pubKey.VerifySignature(challenge, signature)
}
```

악의적 피어 탐지 및 차단

1. 피어 행동 모니터링

행동 패턴 분석

```
type PeerBehavior struct {
InvalidMessages int
TimeoutCount int
DisconnectCount int
LastActivity time.Time
TrustScore float64
}

func (pb *PeerBehavior) UpdateScore() {
// 신뢰도 점수 계산
penalty := float64(pb.InvalidMessages*10 + pb.TimeoutCount*5)
pb.TrustScore = math.Max(0, 100.0 - penalty)
}
```

실시간 모니터링

```
func (p *peer) onReceive(streamID byte, bz []byte) {
defer func() {
if r := recover(); r != nil {
p.Logger.Error("Peer panicked", "err", r)
p.onPeerError(p, r) // 패닉 발생 시 피어 차단
}
}()
// 메시지 검증
if err := p.validateMessage(bz); err != nil {
p.behavior.InvalidMessages++
if p.behavior.InvalidMessages > maxInvalidMessages {
p.onPeerError(p, ErrTooManyInvalidMessages)
}
}
}
```

2. 자동 차단 시스템

Bad Peer 관리

```
type BadPeerManager struct {
badPeers map[nodekey.ID]*BadPeerInfo
```

```go
mutex sync.RWMutex
}

type BadPeerInfo struct {
Reason  string
BanTime time.Duration
BannedAt time.Time
Attempts  int
}

func (bpm *BadPeerManager) BanPeer(id nodekey.ID, reason  string, duration time.
bpm.mutex.Lock()
defer  bpm.mutex.Unlock()
bpm.badPeers[id] = &BadPeerInfo{
Reason: reason,
BanTime: duration,
BannedAt: time.Now(),
Attempts: 1,
}
}
```

차단 기준

- **인증 실패**: 잘못된 서명 또는 인증서

- **프로토콜 위반**: 잘못된 메시지 형식
- **과도한 요청**: Rate limiting 위반
- **악의적 행동**: 잘못된 데이터 전송

3. IP 기반 필터링

중복 IP 차단

```
func ConnDuplicateIPFilter() ConnFilterFunc {
return func(cs ConnSet, c net.Conn, ips []net.IP) error {
for _, ip := range ips {
if cs.HasIP(ip) {
return ErrRejected{
conn: c,
err: fmt.Errorf("ip<%v> already connected", ip),
isDuplicate: true,
}
}
}
return nil
}
}
```

지역 기반 필터링

```go
type GeoFilter struct {
allowedCountries  map[string]bool
blockedIPs  map[string]bool
}


func (gf *GeoFilter) FilterConnection(ip net.IP) error {
country := gf.getCountryByIP(ip)
if !gf.allowedCountries[country] {
return ErrBlockedCountry{Country: country}
}
if  gf.blockedIPs[ip.String()] {
return ErrBlockedIP{IP: ip}
}
return  nil
}
```

네트워크 레벨 보안 정책

1. 연결 제한 정책

최대 연결 수 제한

```
[p2p]
max_num_inbound_peers = 40 # 인바운드  연결  제한
```

```
max_num_outbound_peers = 10 # 아웃바운드 연결 제한
allow_duplicate_ip = false # 동일 IP 중복 연결 차단
```

연결 속도 제한

```go
type ConnectionRateLimit struct {
connections map[string][]time.Time
maxPerIP int
timeWindow time.Duration
mutex sync.RWMutex
}

func (crl *ConnectionRateLimit) AllowConnection(ip string) bool {
crl.mutex.Lock()
defer crl.mutex.Unlock()
now := time.Now()
connections := crl.connections[ip]
// 시간 윈도우 내의 연결만 유지
var validConnections []time.Time
for _, connTime := range connections {
if now.Sub(connTime) < crl.timeWindow {
validConnections = append(validConnections, connTime)
}
```

```
}
if len(validConnections) >= crl.maxPerIP {
return false
}
crl.connections[ip] = append(validConnections, now)
return true
}
```

2. 프라이빗 피어 관리

프라이빗 피어 설정

```
func (sw *Switch) AddPrivatePeerIDs(ids []string) error {
for _, id := range ids {
if err := na.ValidateID(id); err != nil {
return err
}
sw.privatePeerIDs[id] = struct{}{}
}
return nil
}
```

프라이빗 피어 보호

```
func (a *addrBook) addAddress(addr, src *na.NetAddr) error {
```

```
// 프라이빗 피어는 주소록에 추가하지 않음
if _, ok := a.privateIDs[addr.ID]; ok {
return ErrAddrBookPrivate{addr}
}
// 프라이빗 피어로부터 받은 주소도 추가하지 않음
if _, ok := a.privateIDs[src.ID]; ok {
return ErrAddrBookPrivateSrc{src}
}
return a.doAddAddress(addr, src)
}
```

3. 네트워크 분할 방지

최소 연결 유지

```
func (r *Reactor) ensurePeers() {
out, in, dial := r.Switch.NumPeers()
totalPeers := out + in
if totalPeers < r.config.MinPeers {
// 긴급 피어 연결 시도
r.emergencyPeerConnection()
}
}
```

```go
func (r *Reactor) emergencyPeerConnection() {
// 시드 노드에 우선 연결
for _, seed := range r.seedAddrs {
if err := r.Switch.DialPeerWithAddress(seed); err == nil {
break
}
}
// 저장된 좋은 피어들에 연결 시도
goodPeers := r.book.GetGoodPeers(10)
for _, peer := range goodPeers {
r.Switch.DialPeerWithAddress(peer)
}
}
```

DoS/

DDoS 방어 메커니즘

1. Rate Limiting

메시지 속도 제한

```go
type MessageRateLimit struct {
rate int // 초당 메시지 수
```

```go
burst int // 버스트 허용량
tokens int // 현재 토큰 수
lastTime time.Time // 마지막 업데이트 시간
mutex sync.Mutex
}

func (mrl *MessageRateLimit) Allow() bool {
mrl.mutex.Lock()
defer mrl.mutex.Unlock()
now := time.Now()
elapsed := now.Sub(mrl.lastTime)
// 토큰 보충
tokensToAdd := int(elapsed.Seconds() * float64(mrl.rate))
mrl.tokens = min(mrl.burst, mrl.tokens + tokensToAdd)
mrl.lastTime = now
if mrl.tokens > 0 {
mrl.tokens--
return true
}
return false
}
```

대역폭 제한

```go
type BandwidthLimiter struct {
sendRate int64 // bytes per second
recvRate int64 // bytes per second
sendBucket int64
recvBucket int64
lastUpdate time.Time
mutex sync.Mutex
}

func (bl *BandwidthLimiter) AllowSend(size int64) bool {
bl.mutex.Lock()
defer bl.mutex.Unlock()
bl.updateBuckets()
if bl.sendBucket >= size {
bl.sendBucket -= size
return true
}
return false
}
```

2. 연결 보호

SYN Flood 방어

```go
type SYNFloodProtection struct {
connections  map[string]int
maxPerIP  int
cleanupTime time.Duration
mutex sync.RWMutex
}

func (sfp *SYNFloodProtection) AllowConnection(ip  string) bool {
sfp.mutex.Lock()
defer  sfp.mutex.Unlock()
count := sfp.connections[ip]
if  count >= sfp.maxPerIP {
return false
}
sfp.connections[ip] = count + 1
return true
}
```

연결 타임아웃 관리

```go
func (mt *MultiplexTransport) acceptPeers() {
for {
conn, err := mt.listener.Accept()
```

```go
if err != nil {
continue
}
// 연결 처리를 별도 고루틴에서 수행 (Head-of-line blocking 방지)
go func(c net.Conn) {
defer func() {
if r := recover(); r != nil {
c.Close()
}
}()
// 타임아웃 설정
c.SetDeadline(time.Now().Add(handshakeTimeout))
// 연결 필터링 및 업그레이드
if err := mt.filterConn(c); err != nil {
c.Close()
return
}
mconn, remotePubKey, err := mt.upgrade(c, nil)
if err != nil {
c.Close()
return
}
// 성공적인 연결을 채널로 전달
```

```
mt.acceptc <- accept{netAddr, mconn, nil}
}(conn)
}
}
```

3. 리소스 보호

메모리 사용량 제한

```
type MemoryGuard struct {
maxMemory  int64
currentUsage  int64
mutex sync.RWMutex
}

func (mg *MemoryGuard) Allocate(size  int64) bool {
mg.mutex.Lock()
defer  mg.mutex.Unlock()
if  mg.currentUsage + size > mg.maxMemory {
return false
}
mg.currentUsage += size
return true
}
```

```go
func (mg *MemoryGuard) Release(size int64) {
mg.mutex.Lock()
defer mg.mutex.Unlock()
mg.currentUsage -= size
if mg.currentUsage < 0 {
mg.currentUsage = 0
}
}
```

CPU 사용량 모니터링

```go
type CPUMonitor struct {
threshold float64
samples []float64
index int
mutex sync.RWMutex
}

func (cm *CPUMonitor) IsOverloaded() bool {
cm.mutex.RLock()
defer cm.mutex.RUnlock()
var sum float64
```

```
count := 0
for _, sample := range cm.samples {
if sample > 0 {
sum += sample
count++
}
}
if count == 0 {
return false
}
avgCPU := sum / float64(count)
return avgCPU > cm.threshold
}
```

보안 설정 및 모범 사례

1. 보안 설정 옵션

기본 보안 설정

```
[p2p]
# 연결 보안
allow_duplicate_ip = false
max_num_inbound_peers = 40
```

```
max_num_outbound_peers = 10


# 속도 제한
send_rate = 5120000 # 5MB/s
recv_rate = 5120000 # 5MB/s
flush_throttle_timeout = "100ms"


# 메시지 크기 제한
max_packet_msg_payload_size = 1024


# 핸드셰이크 타임아웃
handshake_timeout = "20s"
dial_timeout = "3s"


# 프라이빗 피어 (선택사항)
private_peer_ids = "peer1,peer2,peer3"
```

고급 보안 설정

```
[p2p]
# 연결 필터링
addr_book_strict = true
pex = true
```

```
# 시드 모드 (시드 노드용)
seed_mode = false
seed_disconnect_wait_period = "2m"

# 지속적 피어
persistent_peers = "validator1@10.0.0.1:26656"
persistent_peers_max_dial_period = "0s"

# 무조건적 피어 (연결 제한 무시)
unconditional_peer_ids = "trusted_peer1,trusted_peer2"
```

2. 네트워크 보안 모범 사례

방화벽 설정

```
# CometBFT P2P 포트만 허용
sudo ufw allow 26656/tcp

# RPC 포트는 신뢰할 수 있는 IP만 허용
sudo ufw allow from 192.168.1.0/24 to any port 26657

# 기본 정책: 거부
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

TLS 프록시 사용 (선택사항)

```
upstream cometbft_backend {
server 127.0.0.1:26656;
}

server {
listen 443 ssl;
server_name node.example.com;
ssl_certificate /path/to/cert.pem;
ssl_certificate_key /path/to/key.pem;
location / {
proxy_pass http://cometbft_backend;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
}
}
```

3. 모니터링 및 알림

보안 메트릭 모니터링

```
type SecurityMetrics struct {
```

```go
rejectedConnections prometheus.CounterVec
bannedPeers prometheus.GaugeVec
authFailures prometheus.CounterVec
rateLimitHits prometheus.CounterVec
}

func (sm *SecurityMetrics) RecordRejection(reason  string) {
sm.rejectedConnections.WithLabelValues(reason).Inc()
}

func (sm *SecurityMetrics) RecordBan(peerID  string, reason  string) {
sm.bannedPeers.WithLabelValues(peerID, reason).Set(1)
}
```

보안 이벤트 로깅

```go
type SecurityLogger struct {
logger log.Logger
}

func (sl *SecurityLogger) LogSecurityEvent(event SecurityEvent) {
sl.logger.Error("Security Event",
"type", event.Type,
```

```
"peer", event.PeerID,
"ip", event.IP,
"reason", event.Reason,
"timestamp", event.Timestamp,
)
// 심각한 보안 이벤트는 별도 알림
if event.Severity == "critical" {
sl.sendAlert(event)
}
}
```

4. 보안 감사 및 테스트

정기 보안 점검

```
#!/bin/bash
# 보안 점검 스크립트

echo "=== CometBFT 보안 점검 ==="

# 1. 연결된 피어 수 확인
peer_count=$(curl -s http://localhost:26657/net_info | jq '.result.n_peers')
echo "연결된 피어 수: $peer_count"
```

```
# 2. 차단된 피어 확인
banned_peers=$(curl -s http://localhost:26657/dump_consensus_state | jq '.res
echo "차단된 피어 수: $banned_peers"

# 3. 메모리 사용량 확인
memory_usage=$(ps -o pid,vsz,rss,comm -p $(pgrep cometbft))
echo "메모리 사용량: $memory_usage"

# 4. 로그에서 보안 이벤트 확인
security_events=$(grep -c "Security Event" ~/.cometbft/logs/cometbft.log)
echo "보안 이벤트 수: $security_events"
```

침투 테스트

```
// 보안 테스트 도구
type SecurityTester struct {
targetAddr string
testCases []TestCase
}

type TestCase struct {
Name string
Description string
```

```go
TestFunc func() error
}

func (st *SecurityTester) RunTests() {
for _, tc := range st.testCases {
fmt.Printf("실행 중: %s\n", tc.Name)
if err := tc.TestFunc(); err != nil {
fmt.Printf("실패: %s - %v\n", tc.Name, err)
} else {
fmt.Printf("성공: %s\n", tc.Name)
}
}
}

// 예시 테스트: 과도한 연결 시도
func (st *SecurityTester) testConnectionFlood() error {
for i := 0; i < 1000; i++ {
conn, err := net.Dial("tcp", st.targetAddr)
if err != nil {
return fmt.Errorf("연결 실패: %v", err)
}
conn.Close()
}
```

```
return  nil

}
```

이러한 다층적인 보안 메커니즘을 통해 CometBFT P2P 네트워크는 다양한 보안 위협으로부터 시스템을 보호하고, 안전하고 신뢰할 수 있는 분산 네트워크 환경을 제공합니다.